

Introducción a Event Sourcing

**De aplicaciones CRUD a aplicaciones auditables y
resilientes en Gleam**

Eventsourcing: Una Biblioteca de Event Sourcing

¿Qué es Event Sourcing? 🤔

En lugar de guardar el **estado actual**, guardamos la **secuencia de eventos** que llevaron a ese estado.

CRUD Tradicional:

```
| Balance: $1000 | ← Solo el estado actual
```

Event Sourcing:

```
| 1. AccountOpened |  
| 2. CustomerDepositedCash($500) |  
| 3. CustomerDepositedCash($600) |  
| 4. CustomerWithdrewCash($100) |  
| = Balance: $1000 |
```

CRUD vs Event Sourcing

CRUD Tradicional

-  Simple
-  Pierde historia
-  Difícil auditoría
-  No hay "deshacer"
-  Conflictos de concurrencia

Event Sourcing

-  Historia completa
-  Auditoría total
-  Time-travel debugging
-  Reproducir eventos
-  Múltiples vistas

¿Por qué event sourcing?

Auditabilidad Total

- Cada cambio queda registrado
- Quién, cuándo y por qué

Debugging Mejorado

- Reproducir bugs desde eventos históricos
- Time-travel para entender el problema

Resiliencia del Sistema

- Reconstruir estado desde cero
- Múltiples read models independientes

Conceptos Fundamentales

1. Commands (Comandos)

Intenciones de cambio - pueden fallar

```
pub type BankAccountCommand {  
    OpenAccount(account_id: String)  
    DepositMoney(amount: Float)  
    WithdrawMoney(amount: Float)  
}
```

Conceptos Fundamentales

2. Events (Eventos)

Hechos inmutables - ya sucedieron

```
pub type BankAccountEvent {  
    AccountOpened(account_id: String)  
    CustomerDepositedCash(amount: Float, balance: Float)  
    CustomerWithdrewCash(amount: Float, balance: Float)  
}
```

Conceptos Fundamentales

3. **Aggregate** (Agregado)

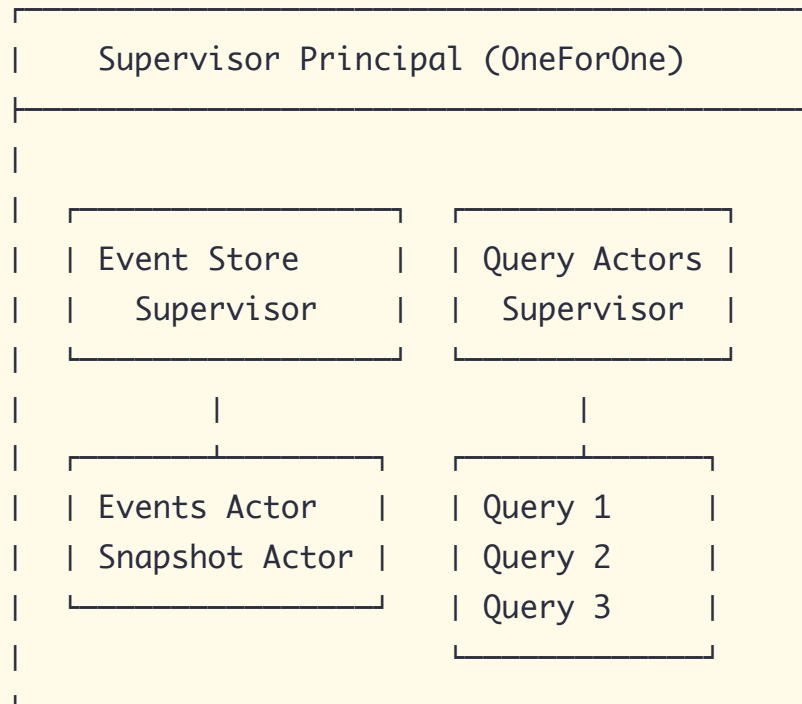
Estado reconstruido desde eventos

```
pub type BankAccount {  
    BankAccount(balance: Float)  
    UnopenedBankAccount  
}
```

4. **Event Store**

Base de datos de eventos (immutable, append-only)

Arquitectura: OTP Supervision Trees



Procesamiento Asíncrono ⚡

Commands → No bloqueantes

- Ejecutan y retornan resultado de forma asíncrona
- Validación de reglas de negocio
- Pueden fallar con errores del dominio

Queries → No bloqueantes

- Procesan eventos en actores separados
- Actualizan read models independientemente
- No afectan el procesamiento de comandos

Filosofía: "Let It Crash"

Errores del Sistema → Crash + Restart

- Problemas de red, base de datos
- Actor se reinicia automáticamente

Errores del Dominio → No Crash

- Validaciones de negocio fallidas
- Se devuelven como `Result(_, Error)`
- El sistema sigue funcionando

Ejemplo: Cuenta Bancaria

Definición del Dominio

```
pub type BankAccount {  
    BankAccount(balance: Float)  
    UnopenedBankAccount  
}  
  
pub type BankAccountError {  
    CantDepositNegativeAmount  
    CantOperateOnUnopenedAccount  
    CantWithdrawMoreThanCurrentBalance  
}
```

Handler de Comandos

```
pub fn handle(
  bank_account: BankAccount,
  command: BankAccountCommand,
) -> Result(List(BankAccountEvent), BankAccountError) {
  case bank_account, command {
    UnopenedBankAccount, OpenAccount(account_id) ->
      Ok([AccountOpened(account_id)])

    BankAccount(balance), DepositMoney(amount) -> {
      case amount >. 0.0 {
        True -> {
          let new_balance = balance +. amount
          Ok([CustomerDepositedCash(amount, new_balance)])
        }
        False -> Error(CantDepositNegativeAmount)
      }
    }
  }
  // ... más casos
}
```

Aplicación de Eventos

Reconstruir el estado desde eventos

```
pub fn apply(  
  bank_account: BankAccount,  
  event: BankAccountEvent  
) -> BankAccount {  
  case event {  
    AccountOpened(_) ->  
      BankAccount(0.0)  
  
    CustomerDepositedCash(_, balance) ->  
      BankAccount(balance)  
  
    CustomerWithdrewCash(_, balance) ->  
      BankAccount(balance)  
  }  
}
```

Flujo Completo

1. Cliente envía Command
- ↓
2. Validar Command con estado actual
- ↓
3. Generar Events si es válido
- ↓
4. Persistir Events en Event Store
- ↓
5. Aplicar Events al estado
- ↓
6. Notificar Query Actors (async)
- ↓
7. Actualizar Read Models

Uso Supervisado (Recomendado) 🎮

```
pub fn main() {  
  // 1. Crear Event Store con supervisión  
  let #(eventstore, memory_store_spec) =  
    memory_store.supervised(...)
```

Uso Supervisado (Recomendado) 🎮

```
// 2. Crear sistema de event sourcing supervisado
let assert Ok(eventsourcing_spec) =
  eventsourcing.supervised(
    eventstore: eventstore,
    ...
  )

// 3. Iniciar supervisor
let assert Ok(_supervisor) = static_supervisor.new(OneForOne)
  |> static_supervisor.add(eventsourcing_spec)
  |> static_supervisor.add(memory_store_spec)
  |> static_supervisor.start()
```


Ejecutar Comandos

```
// Obtener el actor por nombre
let event sourcing = process.named_subject(name)

// Ejecutar comandos
event sourcing.execute(
  event sourcing,
  "account-123",
  OpenAccount("account-123")
)

event sourcing.execute(
  event sourcing,
  "account-123",
  DepositMoney(100.0)
)
```

Comandos con Metadata

Agregar información de rastreo

```
eventsourcing.execute_with_metadata(  
  eventsourcing,  
  "account-123",  
  DepositMoney(100.0),  
  [  
    #("user_id", "user-456"),  
    #("source", "mobile_app"),  
    #("trace_id", "abc-123"),  
    #("ip_address", "192.168.1.1")  
  ]  
)
```

Útil para: auditoría, compliance, debugging

API Asíncrona

```
// Cargar agregado de forma asíncrona
let load_subject = eventsourcing.load_aggregate(
  eventsourcing,
  "account-123"
)

case process.receive(load_subject, 1000) {
  Ok(Ok(aggregate)) ->
    io.println("Cuenta cargada: " <> aggregate.aggregate_id)

  Ok(Error(eventsourcing.EntityNotFound)) ->
    io.println("Cuenta no encontrada")

  Error(_) ->
    io.println("Timeout esperando respuesta")
}
```

Snapshots para Optimización

¿Problema?

- Miles de eventos → lento reconstruir estado

¿Solución?

- Guardar "foto" del estado cada N eventos
- Cargar snapshot + eventos posteriores

```
// Crear snapshot cada 100 eventos
let assert Ok(frequency) = eventsourcing.frequency(100)
let snapshot_config = eventsourcing.SnapshotConfig(frequency)

let assert Ok(spec) = eventsourcing.supervised(
  // ... otros parámetros ...
  snapshot_config: Some(snapshot_config)
)
```

Cargar Snapshot más Reciente

```
let snapshot_subject =
  event sourcing.latest_snapshot(
    event sourcing,
    "account-123"
  )

case process.receive(snapshot_subject, 1000) {
  Ok(Ok(Some(snapshot))) => {
    io.println(
      "Usando snapshot desde secuencia "
      <> int.to_string(snapshot.sequence)
    )
    // Cargar solo eventos después del snapshot
  }
  Ok(Ok(None)) => {
    io.println("Sin snapshot, cargando desde eventos")
  }
}
```

Queries Asíncronos

Read Models actualizados en tiempo real

```
let balance_query = fn(aggregate_id, events) {  
  // Actualizar vista de saldos  
  io.println(  
    "Cuenta " <> aggregate_id <> " procesó "  
    <> int.to_string(list.length(events)) <> " eventos"  
  )  
  
  // Actualizar base de datos de lectura  
  update_read_model(aggregate_id, events)  
}  
  
let queries = [  
  #(process.new_name("balance_query"), balance_query)  
]
```

Event Stores Disponibles

1. In-Memory Store

- Para desarrollo y testing
- Rápido y simple
- No persistente

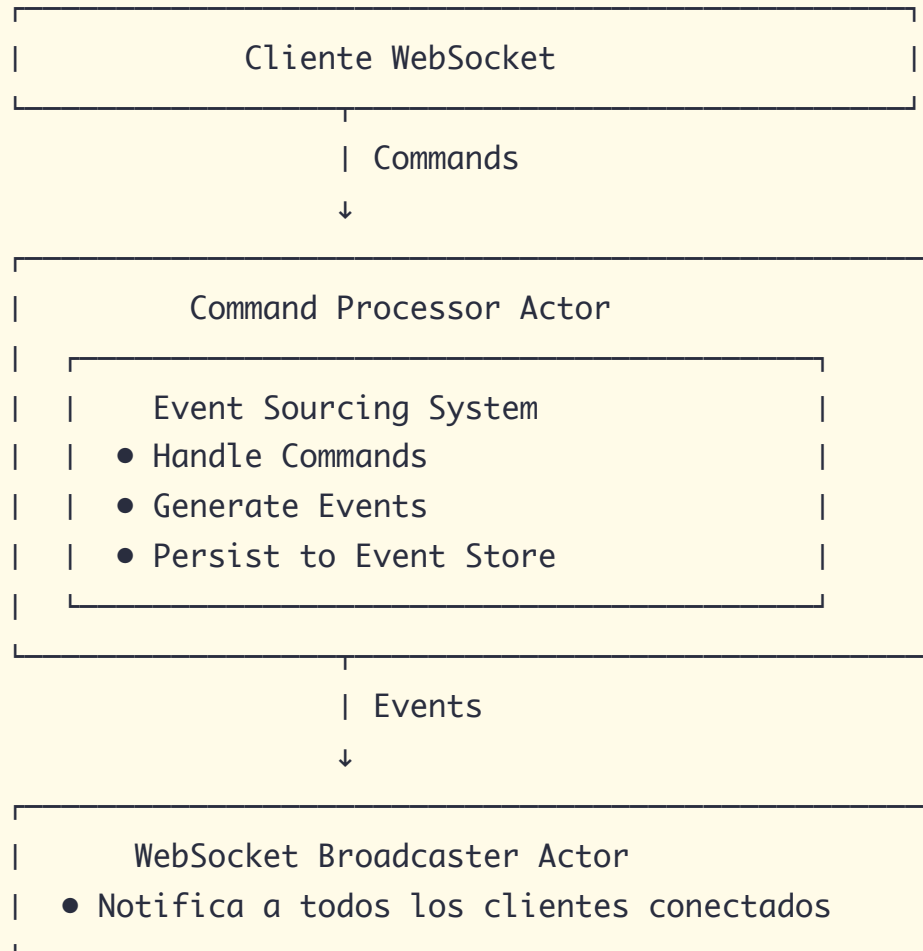
2. PostgreSQL Store

- Producción
- Transacciones ACID
- Alta disponibilidad

3. SQLite Store

- Embedded
- Sin servidor
- Perfecto para edge computing

Ejemplo: Aplicacion con WebSockets



Ventajas de Event Sourcing

Auditabilidad Completa

- Cada cambio registrado para siempre
- Compliance automático

Debugging Superior

- Reproducir cualquier bug
- Time-travel al estado exacto

Resiliencia

- Reconstruir sistema desde eventos
- Múltiples read models

Ventajas de Event Sourcing

Business Intelligence

- Datos históricos completos
- Analytics precisos

Event-Driven Architecture

- Integración con microservicios
- Pub/Sub patterns

Consideraciones

Complejidad

- Curva de aprendizaje mayor
- Más código que CRUD

Eventual Consistency

- Las queries se actualizan de forma asíncrona
- No siempre refleja el último estado

Consideraciones

Almacenamiento

- Los eventos crecen infinitamente
- Necesitas estrategia de archivado

No es para Todo

- CRUD simple → no lo necesitas
- Alta escritura de datos → puede ser complejo
- Considera tu caso de uso

Cuándo Usar Event Sourcing

✓ Usa Event Sourcing si...

- Necesitas auditoría completa
- El historial es importante
- Sistema financiero/legal
- Debugging complejo
- Múltiples vistas de los datos

✗ No uses Event Sourcing si...

- CRUD simple es suficiente
- No necesitas historial
- Prototipo rápido
- Equipo sin experiencia

Testing con Event Sourcing

```
pub fn deposit_money_test() {  
  // Given: Una cuenta con $100  
  let account = BankAccount(100.0)  
  
  // When: Depositamos $50  
  let result = handle(account, DepositMoney(50.0))  
  
  // Then: Debería generar evento correcto  
  let assert Ok([CustomerDepositedCash(50.0, 150.0)]) = result  
  
  // And: El estado debería ser $150  
  let new_account = apply(account, CustomerDepositedCash(50.0, 150.0))  
  let assert BankAccount(150.0) = new_account  
}
```

Testing con Event Sourcing

```
pub fn cant_withdraw_more_than_balance_test() {  
    let account = BankAccount(50.0)  
  
    let assert Error(CantWithdrawMoreThanCurrentBalance) = handle(account, WithdrawMoney(100.0))  
}
```

Fácil testear lógica de negocio: funciones puras

Recursos y Referencias

Documentación

- [Hex.pm Package](#)
- [HexDocs](#)
- [GitHub Repository](#)

Event Stores

- [eventsourcing_postgres](#)
- [eventsourcing_sqlite](#)
- [eventsourcing_glyn](#)

Instalación

```
# Agregar a tu proyecto Gleam
```

```
gleam add eventsourcing
```

```
# PostgreSQL
```

```
gleam add eventsourcing_postgres
```

```
# SQLite
```

```
gleam add eventsourcing_sqlite
```

```
# Glyn (Una libreria wrapper de Syn)
```

```
gleam add eventsourcing_glyn
```

Puntos Clave para Recordar

1. **Eventos = Historia inmutable** de tu sistema
2. **Commands = Intenciones**, Events = Hechos
3. **Supervision Trees** = Resiliencia automática
4. **CQRS** integrado = Escritura y lectura separadas
5. **Snapshots** = Optimización sin perder historia
6. **Queries asíncronos** = No bloquean comandos
7. **Metadata** = Auditoría y rastreo completo

¡Gracias! 🙏

Event Sourcing con Gleam

Construye sistemas resilientes y auditables

¿Preguntas?